

Curso de Especialización

Ciberseguridad en las TIC

Ejercicio 8

Objetivos

- Identificar vulnerabilidades
- Explicar cómo podrían explotarse
- Proponer controles de seguridad adecuados

Resumen

Autora: Marina del Pilar López Oliva

Febrero 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	17/02/2026	Realización del ejercicio.
1.1	18/02/2026	Finalización del ejercicio.

Índice de Contenidos

Identificación de vulnerabilidades.....	4
Explotación de vulnerabilidades.....	4
Inyección SQL.....	4
Credenciales en texto plano.....	4
Falta de autenticación y autorización (IDOR).....	5
No tiene control de privilegios.....	5
Debug mode habilitado en producción.....	5
Controles de seguridad propuestos.....	5
Mejora 1: Inyección SQL.....	5
Mejora 2: Credenciales en texto plano.....	6
Mejora 3: Falta de autenticación y autorización (IDOR).....	6
Mejora 4: No tiene control de privilegios.....	6
Mejora 5: Debug mode habilitado en producción.....	6
Conclusiones de la mejora.....	6
Código mejorado.....	7

Identificación de vulnerabilidades

La aplicación "API REST" proporciona unas tres vulnerabilidades graves. Aunque hay algunas vulnerabilidades más, estas tres son las más graves.

- Inyección SQL: `query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"`. Esta línea es vulnerable a ataques de SQL Injection. Permite a los atacantes manipular las consultas que la aplicación realiza a su base de datos.
- Credenciales en texto plano. Las contraseñas se comparan directamente sin hash. Si la BD se compromete, todas las contraseñas quedan expuestas. Además, se comparan contraseñas en texto plano en la consulta SQL.
- Falta de autenticación y autorización: cualquiera puede acceder a información de cualquier usuario sin estar autenticado. No hay verificación de sesión ni control de acceso.
- No tiene control de privilegios. Cualquier usuario puede cambiar el rol de cualquier otro usuario sin autenticación. No hay verificación de permisos ni validación de roles permitidos.
- Debug mode habilitado en producción: `app.run(debug=True)`. Expone información sensible y permite ejecutar código Python arbitrario mediante el debugger interactivo.

Con estas vulnerabilidades el código es demasiado vulnerable para los atacantes. Un atacante puede ver, modificar y borrar todos los datos, de la base de datos y de la web, sin esforzarse demasiado.

Explotación de vulnerabilidades

Inyección SQL

Un atacante puede inyectar código SQL malicioso. Por ejemplo, introduciendo en la consulta `admin' OR '1=1'`. Esto puede mostrarnos a todos los usuarios. O incluso pueden introducir `; DROP TABLE user;--` para eliminar las tablas correspondientes.

Credenciales en texto plano

Si un atacante obtiene acceso a la base de datos, puede leer todas las contraseñas directamente. También permite ataques de fuerza bruta más eficientes.

Falta de autenticación y autorización (IDOR)

Un atacante puede enumerar IDs de usuarios y obtener información sensible de todos los usuarios sin necesidad de credenciales. Ejemplo: GET /user/1, /user/2... para obtener datos de todos los usuarios.

No tiene control de privilegios

Un atacante puede elevar privilegios de cualquier usuario (incluido al que tiene acceso) a "admin". Por ejemplo:

POST /update_role

```
{"user_id":1, "role": "admin"}
```

Esto permite una escalada de privilegios completa.

Debug mode habilitado en producción

Si hay un error, Flask muestra un stack trace completo que puede revelar:

- Estructura de directorios del servidor.
- Rutas de archivos del sistema.
- Variables de entorno.
- Código fuente completo.

Además, el debugger interactivo permite ejecutar código Python en el servidor.

Controles de seguridad propuestos

La versión mejorada de la aplicación incorpora controles de seguridad fundamentales para mitigar vulnerabilidades que habían presentes en el código original. A continuación se detallan las mejoras aplicadas y su impacto.

Mejora 1: Inyección SQL

Usar consultas parametrizadas.

El uso de ? como parámetro separa la estructura SQL de los datos. SQLite trata el parámetro como valor literal, no como código ejecutable. Se elimina la posibilidad de modificar la consulta mediante entrada maliciosa.

Se mitiga completamente la vulnerabilidad SQL Injection.

Mejora 2: Credenciales en texto plano

Como mejora para este problema se propone utilizar bycript o argon2 para hashear las contraseñas.

Mejora 3: Falta de autenticación y autorización (IDOR)

Para esta solución, podemos hacer uso de las siguientes recomendaciones:

- Requerir autenticación mediante tokens o sesiones.
- Verificar que el usuario solo pueda acceder a sus propios datos.
- Usar bibliotecas como flask-login.

Mejora 4: No tiene control de privilegios

Para solucionar esto, tenemos varias opciones, para poner todas o las que se crean necesarias en este caso:

- Requerir autenticación y verificar que el usuario tenga permisos de administrador.
- Validar que el rol sea uno de los permitidos por la aplicación.
- Implementar un control de acceso basado en roles.

Ejemplo de la implementación de un control de acceso:

```
@require_auth
```

```
@require_role("admin")
```

```
def update_role():
```

```
    allowed_roles = ["user", "admin", "moderator"]
```

```
    if new_role not in allowed_roles:
```

```
        return jsonify({"error": "Rol invalido"}), 400
```

Mejora 5: Debug mode habilitado en producción

La solución más fácil sería deshabilitar el debug en producción: `app.run(debug=False)`. O mejor aún, usar un servidor WSGI en producción.

Conclusiones de la mejora

En conjunto, estas medidas reducen significativamente las vulnerabilidades críticas presentes en la versión original y fortalecen la seguridad de la aplicación frente a ataques comunes.

Código mejorado

```
from flask import Flask, request, jsonify, g

import sqlite3

import hashlib

import hmac

import os

import functools

app = Flask(__name__)

# --- Configuración para el token y roles para require_auth y requiere_role---

SECRET_ADMIN_TOKEN = os.environ.get("ADMIN_TOKEN",
    "t0kenS3guroAdm1n_T0ken")

ROLES_VALIDOS = {"user", "admin", "moderator"}

# --- Base de datos ---

def get_db():

    conn = sqlite3.connect("api_users.db")

    conn.row_factory = sqlite3.Row

    return conn

# --- Autenticación por token simple (utilización del SECRET_ADMIN_TOKEN) ---

def require_auth(f):

    # Verifica que la petición incluya un token válido.

    @functools.wraps(f)
```

Ejercicio 8

```
def wrapper(*args, **kwargs):
    token = request.headers.get("Authorization", "").replace("Bearer ", "")
    if not token:
        return jsonify({"error": "No autorizado"}), 401

    conn = get_db()
    user = conn.execute(
        "SELECT * FROM users WHERE session_token = ?", (token,)
    ).fetchone()
    conn.close()

    if not user:
        return jsonify({"error": "Token inválido"}), 401

    g.current_user = dict(user)
    return f(*args, **kwargs)

return wrapper


def require_role(role):
    # Verifica que el usuario autenticado tenga el rol requerido (utilización de
    ROLES_VALIDOS).

    def decorator(f):
        @functools.wraps(f)
        @require_auth
        def wrapper(*args, **kwargs):
            if g.current_user.get("role") != role:
```


Ejercicio 8

```
        return jsonify({"error": "Permisos insuficientes"}), 403

    return f(*args, **kwargs)

    return wrapper

    return decorator


# --- Utilidad: hash de contraseña ---

def hash_password(password: str) -> str:

    # Hashea la contraseña con SHA-256. En producción usar bcrypt/argon2.

    return hashlib.sha256(password.encode()).hexdigest()


# --- Endpoints ---


@app.route("/login", methods=["POST"])
def login():

    data = request.json or {}

    username = data.get("username", "").strip()
    password = data.get("password", "")

    if not username or not password:

        return jsonify({"error": "Faltan credenciales"}), 400

    conn = get_db()

    cursor = conn.cursor()
```

Ejercicio 8

CORREGIDO: no se usaban parámetros en el enunciado.

Forma correcta: pasar query y params por separado.

```
user = cursor.execute(
```

```
    "SELECT * FROM users WHERE username = ?", (username,)
```

```
).fetchone()
```

```
conn.close()
```

CORREGIDO: comparar el hash de la contraseña, nunca texto plano.

Se usa hmac.compare_digest para evitar timing attacks.

```
if user and hmac.compare_digest(user["password"], hash_password(password)):
```

```
    return jsonify({"message": "Login correcto", "user_id": user["id"], "role":
user["role"]})
```

Mensaje genérico: no revelar si el usuario existe o no.

```
return jsonify({"message": "Credenciales inválidas"}), 401
```

```
@app.route("/user/<int:user_id>", methods=["GET"])
```

```
@require_auth
```

```
def get_user(user_id):
```

```
    current = g.current_user
```

CORREGIDO: control de autorización.

Un usuario solo puede ver su propio perfil, salvo que sea admin.

```
if current["id"] != user_id and current.get("role") != "admin":
```

```
    return jsonify({"error": "No autorizado para ver este usuario"}), 403
```

Ejercicio 8

```
conn = get_db()

user = conn.execute(
    "SELECT id, username, role FROM users WHERE id = ?", (user_id,)
).fetchone() # CORREGIDO: nunca devolver la contraseña en la respuesta.

conn.close()
```

```
if user:
    return jsonify(dict(user))

return jsonify({"error": "No encontrado"}), 404
```

```
@app.route("/update-role", methods=["POST"])
@require_role("admin") # CORREGIDO: solo admins pueden cambiar roles.
```

```
def update_role():
```

```
    data = request.json or {}
    user_id = data.get("user_id")
    new_role = data.get("role", "").strip()
```

```
# CORREGIDO: validar que el rol sea uno de los permitidos.
```

```
if not user_id or new_role not in ROLES_VALIDOS:
```

```
    return jsonify({"error": f"Rol inválido. Opciones: {ROLES_VALIDOS}"}), 400
```

```
# CORREGIDO: un admin no puede quitarse su propio rol (protección extra).
```

```
if g.current_user["id"] == user_id:
```

```
    return jsonify({"error": "No puedes modificar tu propio rol"}), 403
```

```
conn = get_db()
```

Ejercicio 8

```
cursor = conn.cursor()
cursor.execute("UPDATE users SET role = ? WHERE id = ?", (new_role, user_id))
conn.commit()
conn.close()
```

```
return jsonify({"message": f"Rol actualizado a '{new_role}'"})
```

```
if __name__ == "__main__":
```

```
    # CORREGIDO: debug=True nunca en producción (expone el debugger interactivo).
```

```
    app.run(debug=False)
```